Rede sobreposta com encaminhamento

Redes de Computadores e Internet
2º Semestre 2025/2026
Projeto de Laboratório, versão 1

1. Introdução

Pretende-se desenvolver uma aplicação que permite criar *redes sobrepostas* à internet com um serviço de *encaminhamento* incluído. Uma aplicação de *chat* permitirá testar as redes sobrepostas e o seu serviço de encaminhamento.

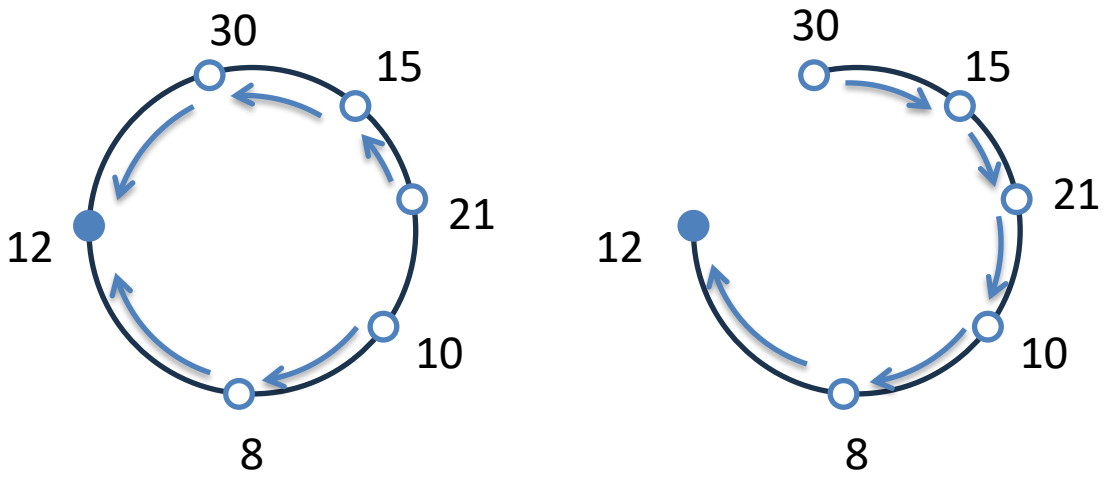
Uma rede de sobreposição é constituída por *nós com identificadores únicos*, cada um deles uma instância da aplicação, e *arestas*, substanciadas em sessões TCP entre dois nós. A comunicação sobre cada aresta é bidirecional, existindo no máximo uma aresta entre cada par de nós. Um servidor de nós (fornecido pelo corpo docente) gere uma base-de-dados dinâmica com as correspondências entre os identificadores dos nós e os seus *contactos*, sendo que o contacto de um nó (na internet) consiste no seu endereço IP e porto TCP. A comunicação com o servidor de nós dá-se por UDP. Um nó que participe na rede, e só neste caso, tem o seu contacto registado no servidor de nós. Ele pode consultar o servidor de nós para conhecer os identificadores de todos os outros nós na rede e também o pode consultar para obter o contacto de um outro nó através do identificador deste. Ele pode estabelecer uma aresta com outro nó e remover uma aresta previamente estabelecida.

É bem possível, aliás muito provável, que a rede de sobreposição não seja completa, isto é, que nem todos os pares de nós tenham uma aresta entre si. Numa rede incompleta levanta-se a questão de como garantir que um dado nó comunica com outro. Poder-se-ia assegurar que cada mensagem fosse difundida por toda a rede, mas claramente essa forma de comunicação desperdiça recursos de transmissão na rede. A alternativa é encaminhar cada mensagem ao longo de um único caminho da sua origem para o seu destino. Vamos optar por *encaminhamento por caminho mais curto*, com o comprimento do caminho medido pelo número de arestas que contém. Com este propósito, os nós executam um *protocolo de encaminhamento*, isto é, um algoritmo distribuído que lhes permite determinar, para cada destino e a cada instante, *vizinhos de expedição*, que aproximam as mensagens recebidas dos seus destinos. O protocolo de encaminhamento deverá reagir automaticamente aos acontecimentos seguintes: (i) anúncio de um novo nó; (ii) introdução de uma nova aresta; (iii) remoção de uma aresta. Na ausência de acontecimentos dos tipos elencados acima, os vizinhos de expedição deverão guiar as mensagens ao longo de caminhos mais curtos. Estudará no curso um par de protocolos de encaminho que satisfazem os requisitos anteriormente especificados. Contudo, o protocolo de encaminhamento que vai implementar neste projeto é especial no seguinte requisito: em nenhum momento durante a execução do protocolo os vizinhos de

expedição definem um *ciclo de expedição*, sobre os qual as mensagens ficariam a circular. A secção seguinte apresenta as regras do protocolo e ilustra o seu comportamento.

Para testar a rede de sobreposição e o serviço de encaminhamento, desenvolverá também uma simples aplicação *chat* que envia mensagens de um nó para outro.

2. Protocolo de encaminhamento sem ciclos de expedição



O encaminhamento tem como objetivo selecionar caminhos mais curtos e guiar as mensagens de dados ao longo desses caminhos. Por exemplo, na rede da figura, que é em anel, o caminho mais curto do nó 15 para o nó destino 12 é o caminho 15-30-12. Cada mensagem inclui o identificador do seu destino no cabeçalho. Cada nó mantém um vizinho de expedição para o destino, sendo este o vizinho que aproxima as mensagens de dados do destino ao longo de caminhos mais curtos. Por exemplo, relativamente ao destino 12: o vizinho de expedição do nó 30 é o 12; o do nó 15 é o 30; o do nó 21 é o 15; o do nó 10 é o 8; e, por último, o do nó 8 é o 12. Uma mensagem originada no nó 21 com destino no nó 12 é expedida no nó 21 para o nó 15, daí é expedida para o nó 30, o qual a entrega ao nó 12.

Suponha agora que a ligação 12-30 falha. O nó 30 deteta a falha (e também o nó 12, mas isso não é relevante). Se ele mudar o seu vizinho de expedição para o nó 15 logo que deteta a falha, então estabelece-se um ciclo de expedição entre os nós 30 e 15. Para que não se criem ciclos de expedição, o protocolo de encaminhamento deverá assegurar que as atualizações dos vizinhos de expedição dos nós sejam coordenadas. Voltando ao exemplo anterior, o protocolo de encaminhamento deverá assegurar que: o nó 21 seja o primeiro a trocar de vizinho de expedição, para o nó 10; depois o nó 15, para o nó 21; e, por último, o nó 30 que atualizará o vizinho de expedição para o nó 15.

Apresenta-se de seguida um protocolo de encaminhamento que permite a coordenação descrita anteriormente numa rede arbitrária.

2.1 Estado, mensagens e acontecimentos

Cada nó mantém a seguinte informação de estado.

- $dist[t]$ – estimativa da distância a t. Se não for possível alcançar t, então $dist[t] = \infty$.
- $succ$[t] – identificador do vizinho de expedição para t, sendo este o vizinho por onde o nó aprendeu a distância $dist[t]$. Se não for possível alcançar t, então $succ[t] = -1$.
- $state[t]$ – estado relativo a t, tomando o valor 0 no caso do estado de *expedição* e o valor 1 no caso de estado de *coordenação*.
- No estado de coordenação relativo a t, o nó mantém, adicionalmente, as variáveis seguintes.
    - $succ_coord[t] -$ identificador do vizinho de expedição que causou a transição para o estado de coordenação. Se a transição foi devida à falha de ligação com o vizinho de expedição, então $succ_coord[t] = -1$.
    - $coord[t, j]$, para cada j vizinho do nó – coordenação com o nó j, tomando o valor 1 se essa coordenação está em curso e o valor 0 se está terminada.

Há três tipos de mensagens de encaminhamento.

- $(route, t, n)$ – anúncio da distância n ao nó t.
- $(coord, t)$ – início de coordenação respeitante ao nó t.
- $(exped, t)$ – fim de coordenação respeitante ao nó t.

O destino t anuncia-se na rede enviando a mensagem de encaminhamento $(route, t, n)$ a todos os seus vizinhos. Há três tipos de acontecimentos que ativam o protocolo.

- Receção de uma mensagem de encaminhamento.
- Adição de uma aresta.
- Remoção (ou falha) de uma aresta.

2.2 Receção de uma mensagem tipo rota

Suponha que o nó recebe do seu vizinho j a mensagem $(route, t, n)$.

1. Se $n + 1 < dist[t]$, então:
    a. $dist[t] := n + 1$; $succ[t] := j$;
    b. se $state[t] = 0$, então o nó envia $(route, t, dist)$ a todos os seus vizinhos.

2.3 Receção de uma mensagem tipo coordenação

Suponha que o nó recebe do vizinho j a mensagem $(coord, t)$.

1. Se $state[t] = 1$, então envia $(exped, t)$ a j.
2. Se $state[t] = 0$ e $j \neq succ[t]$, então envia $(route, t, dist)$ e $(exped, t)$ a j.

3. Se $state[t] = 0$ e $j = succ[t]$, então $state[t] := 1$; $succ_coord[t] := succ[t]$; $dist[t] := \infty$; $succ[t] := -1$, e envia $(coord, t)$ a todos os seus vizinhos.

2.4 Receção de uma mensagem tipo expedição

Suponha que o nó recebe do seu vizinho j a mensagem $(exped, t)$.

1. Se $state = 1$, então $coord[j] := 0$.
2. Se $coord[j, t] = 0$ para todo o vizinho j, então $state[t] := 0$. Se $dist[t] \neq \infty$, então envia $(route, t, dist)$ a todos vizinhos. Se $succ_coord[t] \neq -1$, então envia $(exped, t)$ a $succ_coord[t]$.

2.5 Adição e remoção de uma aresta

Suponha que um nó adquire um novo vizinho por adição de uma aresta. Se o nó está no estado de expedição relativamente a um destino, então ele deverá enviar uma mensagem de tipo rota, com a distância a esse destino, ao novo vizinho. Se está no estado de coordenação, então ele deverá registar que o seu regresso ao estado de expedição não depende to vizinho.

Suponha que um nó perde um vizinho por remoção (ou falha) de uma aresta. Se o nó está no estado de expedição relativamente a um destino e o seu sucessor é o vizinho perdido, então ele entra no estado de coordenação e segue o procedimento associado e este estado. Se está no estado de coordenação, ele deverá registar que o seu regresso ao estado de expedição não depende to vizinho.

2.6 Exemplo de operação

Considere o nó destino 12 na rede da figura e suponha que ligação 12-30 falha. Porque 12 era o vizinho de expedição do nó 30, este entra no estado de coordenação e envia uma mensagem de coordenação ao nó 15. Ao receber esta mensagem, o nó 15 verifica se ela foi remetida pelo seu vizinho de expedição. Sendo este o caso, o nó 15 entra no estado de coordenação, e envia uma mensagem de coordenação ao nó 21 (também enviará uma ao nó 30 que lhe responderá logo com uma mensagem de expedição). O nó 21 recebe a mensagem de coordenação vinda do seu vizinho de expedição, entra no estado de coordenação e envia uma mensagem de coordenação ao nó 10. O nó 10 recebe a mensagem de coordenação, mas ela não foi remetida pelo seu vizinho de expedição. Então, o nó 10 responde ao nó 21, com a sua distância ao destino, que é 2, e com uma mensagem de expedição. Quando o nó 21 aprende que o nó 10 alcança o nó 12 à distância de 2, atualiza a sua distância a 12 para 3, mas permanece no estado de coordenação; de seguida, quando recebe a mensagem de expedição remetida pelo nó 10, regressa ao estado de expedição, envia a sua distância aos nós 10 (que não fará uso dela) e 15, e envia uma mensagem de expedição ao nó 15. Quando o nó 15 aprender a nova distância de 21 a 12, que agora é 3, atualiza a sua própria distância ao nó 12 para 4; e quando receber a

mensagem de expedição, regressa no estado de expedição, envia a sua distância aos nós 21 e 30 e uma mensagem de expedição ao nó 30. Finalmente, quando este receber as mensagens anteriores, atualiza a sua distância para 5 e seu vizinho de expedição para 15.

3. Especificação

Cada grupo de dois alunos deve concretizar a aplicação **`OWR (OverlayWithRouting)`** correspondendo a um nó num anel. A aplicação **`OWR`** é composta pelos elementos seguintes:

- comando de invocação da aplicação;
- interface de utilizador;
- protocolos de comunicação com servidor de nós;
- protocolo de rede sobreposta;
- protocolo de encaminhamento;
- protocolo *chat*.

3.1 Comando de invocação da aplicação

A aplicação **`OWR`** é invocada com o comando

`OWR` ***`IP TCP regIP regUDP`***

em que ***`IP`*** e ***`TCP`*** constituem o *contacto* do nó criado pela aplicação: ***`IP`*** é o endereço da máquina que aloja a aplicação e ***`TCP`*** é o porto TCP servidor da aplicação (porto de escuta). Os parâmetros ***`regIP`*** e ***`regUDP`*** são o contacto, endereço IP e porto UDP, respetivamente, do servidor de nós fornecido pelo corpo docente. Por omissão, estes parâmetros tomam os valores **`193.136.138.142`** e **`59000`**.

3.2 Interface de utilizador

A interface de utilizador consiste nos seguintes comandos, as abreviaturas dos quais estão entre parêntesis.

- **`join (j)`** ***`net id`***
  Participação de um nó com identificador ***`id`*** na rede ***`net`***. Deverá assegurar-se que o valor ***`id`*** é único na rede ***`net`***, se necessário for interagindo com o servidor de nós através de outros comandos. Os valores de ***`net`*** são representados por três dígitos, podendo variar entre ***`000`*** e ***`999`*** e os valores de ***`id`*** são representados por dois dígitos, podendo variar entre ***`00`*** e ***`99`***. Em consequência deste comando, o nó regista o seu contacto na base-de-dados.
- **`show nodes (n)`** ***`net`***
  Um nó consulta a base-de-dados para obter a lista de identificadores de todos os nós pertencentes à rede ***`net`***.

- **`leave (l)`**
  Um nó deixa de participar na sua rede. O nó remove todas as arestas na rede que incidiam sobre ele e remove o seu contacto da base-de-dados.
- **`exit (x)`**
  Fecho da aplicação. Se o comando não for precedido pelo comando **`leave`**, então este último é invocado agora.
- **`add edge (ae)`** ***`id`***
  Um nó estabelece uma aresta com o nó da sua rede com identificador ***`id`***. Há no máximo uma aresta entre um par de nós.
- **`remove edge (re)`** ***`id`***
  Um nó remove a aresta que partilhava com o nó da sua rede com identificador ***`id`***.
- **`show neighbors (sg)`**
  Um nó apresenta a lista dos seus vizinhos na rede.
- **`announce (a)`**
  Um nó anuncia-se na sua rede para ser alcançável por todos os outros nós da rede.
- **`show routing (sr)`** ***`dest`***
  Um nó apresenta o estado do encaminhamento relativamente ao nó destino ***`dest`***, nomeadamente se está no estado de expedição ou de coordenação, e, no primeiro caso, qual a distância e o vizinho de expedição relativamente ao destino.
- **`start monitor (sm)`** e **`end monitor (em)`**. Por omissão, as mensagens de encaminhamento trocadas com o nó não são mostradas ao utilizador. Com estes comandos, ativa-se e desativa-se a monitorização desses mensagens.
- **`message (m)`** ***`dest message`***
  Envio da mensagem ***`message`*** ao nó ***`dest`***.

Deverá ainda implementar os dois comandos seguintes que permitem criar uma rede sem comunicação com o servidor de nós.

- **`direct join (dj)`** ***`net id`***
  Participação de um nó com identificador ***`id`*** na rede ***`net`*** sem registo no servidor de nós.
- **`direct add edge (dae)`** ***`id idIP idTCP`***
  O nó estabelece uma aresta com o nó da sua rede com identificador ***`id`***, sendo ***`idIP`*** e ***`idTCP`*** o contacto, endereço IP e porto TCP, deste último nó.

3.3 Protocolo de comunicação com o servidor de nós

As mensagens para a comunicação com a base-de-dados são as seguintes.

- **`NODESQUERY`** ***`net`***
  Um nó pede ao servidor de nós os identificadores de todos os nós que participam na rede ***`net`***.
- **`NODESRESP`** ***`net<LF>`***
  `id1 <LF>`

id2 <LF>

…

O servidor de nós envia uma lista com os identificadores de todos os nós na rede ***net***. Cada identificador é enviado numa linha separada.

- **CONTACTQUERY *net id***
  Um nó pede ao servidor de nós o contacto, endereço IP e porto TCP, do nó ***id*** na rede ***net***.
- **CONTACTRESP *net id IP TCP***
  O servidor de nós fornece o contacto do nó ***id*** na rede ***net***.
- **REG *net id IP TCP***
  Um nó regista-se na rede ***net***.
- **OKREG *control_message***
  O servidor de nós responde a um pedido de registo de um nó, oferecendo a mensagem de controlo ***control_message***. A mensagem será **DONE** se o registo tiver sido concretizado e **DUP** se já existir um nó na rede com o mesmo identificador.
- **UNREG *net id***
  O nó retira o seu registo na rede ***net***.
- **OKUNREG *control_message***
  O servidor de nós responde a um pedido de remoção de um registo, oferecendo a mensagem de controlo ***control_message***. A mensagem será **DONE** se o registo tiver sido removido e **NULL** se o nó não estava registado na rede.

3.4 Protocolo de rede sobreposta

O protocolo topológico compreende apenas um tipo de mensagem.

- **NEIGHBOR *id***
  Um nó dá a conhecer a outro o seu identificador ***id***.

3.5 Protocolo de encaminhamento

- **ROUTE *dest n***
  Um nó anuncia a outro que consegue alcançar o destino ***dest*** com ***n*** saltos.
- **COORD *dest***
  Um nó informa um vizinho de que entrou no estado de coordenação relativamente ao destino ***dest***.
- **UNCOORD *dest***
  Um nó informa um vizinho que previamente lhe enviou uma mensagem de coordenação de que não depende desse vizinho para alcançar o destino ***dest***.

3.6 Protocolo de *chat*

Existe apenas um tipo de mensagem de *chat*.

- **`CHAT` *`origin dest chat<LF>`***
  O nó origem ***`origin`*** envia ao nó destino ***`dest`*** a mensagem ***`chat`***. A sequência de caracteres ***`chat`*** tem no máximo 128 caracteres.

4. Desenvolvimento

Cada grupo de alunos deve adquirir a destreza necessária sobre programação em redes para realizar a aplicação proposta.

4.1 Etapas recomendadas

As etapas seguintes são recomendadas na concretização do projeto. Os números em parêntese retos são a duração de referência para cada etapa.

1. Leitura atenta do enunciado do projeto. [um dia]
2. Criação de uma rede com apenas um nó, registo e cancelamento de registo, comandos **`join`** e **`leave`**. [uma semana]
3. Criação de uma rede com vários nós, comandos **`show nodes`**, **`add edge`** e **`show neighbors`**. [uma semana]
4. Anúncio de um nó, comandos **`announce`**, **`show routing`**, **`start monitor`** e **`end monitor`**. [uma semana]
5. Reação do encaminhamento à remoção de uma aresta e à adição de uma aresta, comandos **`add edge`** e **`remove edge`**. [duas semanas, em paralelo com o próximo]
6. Envio de mensagens, comando **`message`**. [duas semanas, em paralelo com o anterior]

4.2 Chamadas de sistema

O código da aplicação fará uso das seguintes primitivas:

- leitura de informação do utilizador para a aplicação: `fgets()`;
- conversão entre *strings* e tipos de dados: `sscanf()`, `sprintf()`;
- gestão de um cliente UDP: `socket()`, `close()`;
- gestão de um servidor UDP: `socket()`, `bind()`, `close()`;
- comunicação UDP: `sendto()`, `recvfrom()`;
- gestão de um cliente TCP: `socket()`, `connect()`, `close()`;
- gestão de um servidor TCP: `socket()`, `bind()`, `listen()`, `accept()`, `close()`;
- comunicação TCP: `write()`, `read()`;
- multiplexagem de entradas síncronas: `select()`.

4.3 Depuração

Faça uso das estratégias seguintes que permitem uma boa depuração do código:

- comente o código à medida que o desenvolve;

- use um depurador, `gdb` ou `xgdb`, para inspecionar a execução sequencial do código num nó;
- use o comando `nc` (`netcat`) para testar a comunicação com um cliente ou com um servidor, quer seja TCP ou UDP;
- execute o analisador de protocolos Wireshark para visualizar todas as mensagens trocadas com um nó e seus conteúdos;
- tente inter-operar um nó implementado pelo seu grupo com um nó implementado por outro grupo (isto é possível porque o formato das mensagens é o mesmo para todos).

Quer os clientes quer os servidores devem terminar graciosamente, pelo menos nas seguintes situações de falha:

- mensagens mal formatadas;
- sessão TCP fechada abruptamente;
- erro nas chamadas de sistema.

5. Entrega do projeto e avaliação

Cada grupo deve submeter no sistema fénix um ficheiro comprimido `.zip` (não é aceite `.rar`) com os três elementos seguintes: (i) código fonte da aplicação **`OWR`**; (ii) a respetiva `makefile`; e (iii) uma ficha de auto-avaliação preenchida que será disponibilizada mais tarde. A execução do comando `make` deverá produzir o executável **`OWR`** sem quaisquer erros ou avisos. O código será testado apenas no ambiente de desenvolvimento do laboratório.

A data de submissão do projeto é sexta-feira, dia 27 de março, até às 23:59. Havendo um qualquer imprevisto, o projeto poderá ser submetido na segunda-feira seguinte, dia 30 de março, até às 23:59, com dois pontos de penalização.

Cada projeto será avaliado numa discussão oral de aproximadamente 20 minutos a ter lugar na semana de 4 de abril, com um calendário que será publicado posteriormente. Ambos os elementos de um grupo devem participar ativamente no projeto, sendo a nota individual a cada aluno. Nas primeiras aulas de laboratório da semana de 9 de março haverá uma avaliação intermédia para três valores. Esta avaliação incidirá sobre a comunicação com o servidor de nós e a criação de uma rede.